

Employee Management System Documentation & User Manual

Akito Yamaguchi, IXF6DR

Introduction

Motivation

In recent years, many companies and organizations have been promoting digitalization in order to improve the efficiency of employee information management and reduce operational workload. Traditionally, employee information has often been managed using paper documents or spreadsheet software; however, these approaches are prone to issues such as update omissions, duplicate data, and poor searchability. In addition, as the number of employees increases, the complexity and cost of data management also become significant challenges.

To address these issues, this project developed an employee management system for centralized management of employee information. The system provides functionalities such as employee registration, editing, deletion, and searching, and was implemented as a desktop application with an intuitive user interface. By doing so, the system aims to improve the efficiency of employee data management while reducing human error and enhancing data consistency.

Furthermore, introducing an employee management system enables faster access to necessary information, contributing to overall operational efficiency. Managing information through a database also improves data persistence and search performance, while making future feature expansion easier. In addition, this project aims to achieve a maintainable and extensible software architecture by adopting a modern technology stack.

Problems addressed

In many organizations, employee information and organizational structures are still managed manually or through spreadsheet software, leading to increasing operational complexity and management overhead. As organizations grow in size, it becomes more difficult to properly manage departmental hierarchies, maintain consistency in employee records, and enforce appropriate access control.

First, organizational structure management requires maintaining accurate relationships and hierarchies between departments. However, manual management methods often make it difficult to efficiently handle structural changes and updates. Similarly, managing position information and employee records manually can result in update omissions, duplicate entries, and data inconsistencies, ultimately reducing operational efficiency.

Furthermore, when salary increases and promotions are handled manually, managing evaluation criteria and

update schedules becomes increasingly complex, increasing the risk of human error. Therefore, there is a need for a system capable of automatically processing promotions and salary adjustments based on predefined rules and conditions.

In addition, audit functionality is an important requirement in enterprise systems, as it enables tracking of who performed specific operations and when those actions occurred. Without proper logging functionality, it becomes difficult to trace unauthorized operations or user mistakes, potentially affecting the reliability and security of the system.

Moreover, allowing all users to have the same level of access creates risks such as unauthorized access to sensitive information and unintended data modification. To address this issue, it is necessary to implement authorization mechanisms based on user types, such as administrators and standard users, in order to enforce proper access control.

This system was developed to address these challenges by providing integrated support for organizational structure management, employee and position data management, automated promotion and salary adjustment processing, audit logging, and role-based authorization.

Functional Requirements

1. Authentication & Authorization

1. Login page

1. Institution name and the service name “Employee Management System” is displayed.
2. Users can login by entering email and password, then click login button.
3. Language can be selected.

2. Sidebar

1. “Users” page in the sidebar is hidden for non-admin users.
2. Global settings button next to the institution name is hidden for non-admin users.
3. User card with username and email is displayed in the bottom of the sidebar.
4. By pressing logout in the user card actions, the user can logout from the system.
5. By pressing change password button in the user card actions, change password modal pops up. By entering current password and new password, password can be changed.
6. By pressing edit profile button in the user card actions, edit profile modal pops up. First name, last name and email can be edited.

3. Users list page

1. User list can be filtered with user name and admin access.
2. By pressing add button, navigate to user registration page.
3. User list has id, first name, last name, email, admin access columns and delete button.
4. User list has 10 rows in each page. Page can be switched with pagination component below the list.
5. By clicking delete button in the user list, a confirmation modal pops up. By pressing confirm, user can be deleted.

4. User registration

1. New user can be registered with first name, last name, email, admin access, password and password confirmation.
2. By pressing confirm, data is validated and new user is registered.
3. By pressing cancel, navigate to previous page.

5. Others

1. Add/edit/delete feature in organization editor is disabled for non-admin users.
2. Add button in department list page is hidden for non-admin users.
3. Edit button in department details page is hidden for non-admin users.
4. Add button in sub-department list page is hidden for non-admin users.
5. Edit button in sub-department details page is hidden for non-admin users.
6. Add button in unit list page is hidden for non-admin users.
7. Edit button in unit details page is hidden for non-admin users.

8. Add button in position list page is hidden for non-admin users.
9. Edit button in position details page is hidden for non-admin users.

2. Settings

1. Institution settings

1. By pressing the setting button on the top of the sidebar, institution setting modal pops up.
Institution name and currency can be edited.

2. Application settings

1. Language can be selected from the user card actions. The supported languages are English, Japanese, Arabic, Hungarian, Chinese and German.
2. Theme (light/dark) can be selected from the user card actions.
3. Primary theme color can be selected from the user card actions.

3. Dashboard

1. Average salary is displayed.
2. Total employee count is displayed.
3. Today's date is displayed.
4. Total department counts is displayed.
5. Employee count by status is displayed as a pie chart.
6. Employee count by department is displayed as a tree map.
7. Daily activities are displayed as an area chart.
8. Total sub-department count is displayed.
9. Total unit count is displayed.
10. Employee count by job grade is displayed as a bar chart.
11. Average salary by department is displayed as a bar chart.
12. Each items in the dashboard can be moved with drag and drop.

4. Departments

1. Department list page

1. Department list can be filtered by name, department code and status.
2. By pressing add button, navigate to department registration page.
3. Department list has name, department code, status columns and open details button.
4. Department list has 10 rows in each page. Page can be switched with pagination component below the list.

2. Department registration page

1. Department form has name, department code, status and description.
2. By pressing confirm button, form data is validated and new department is registered.
3. By pressing cancel button, navigate to previous page.

3. Department details page

1. Department name, code, status and description are displayed.

2. By pressing edit button, the data can be edited, and by pressing confirm, changes are applied. By pressing cancel, changes are discarded.
3. By pressing delete button, the department can be deleted from the system.
4. By pressing go back button, navigate to previous page.

5. Sub Departments

1. Sub-department list page

1. Sub-department list can be filtered by name, sub-department code, status and parent department.
2. By pressing add button, navigate to sub-department registration page.
3. Sub-department list has name, sub-department code, parent department name, status columns and open details button.
4. Sub-department list has 10 rows in each page. Page can be switched with pagination component below the list.

2. Sub-department registration page

1. Sub-department form has name, department code, status, parent department and description.
2. By pressing confirm button, form data is validated and new sub-department is registered.
3. By pressing cancel button, navigate to previous page.

3. Sub-department details page

1. Sub-department name, code, status, parent department and description are displayed.
2. By pressing edit button, the data can be edited, and by pressing confirm, changes are applied. By pressing cancel, changes are discarded.
3. By pressing delete button, the sub-department can be deleted from the system.
4. By pressing go back button, navigate to previous page.

6. Units

1. Unit list page

1. Unit list can be filtered by name, unit code, status and parent sub-department.
2. By pressing add button, navigate to unit registration page.
3. Unit list has name, unit code, parent sub-department, status columns and open details button.
4. Unit list has 10 rows in each page. Page can be switched with pagination component below the list.

2. Unit registration page

1. Unit form has name, unit code, status, parent sub-department and description.
2. By pressing confirm button, form data is validated and new unit is registered.
3. By pressing cancel button, navigate to previous page.

3. Unit details page

1. Unit name, code, status, parent sub-department and description are displayed.
2. By pressing edit button, the data can be edited, and by pressing confirm, changes are applied. By pressing cancel, changes are discarded.
3. By pressing delete button, the unit can be deleted from the system.
4. By pressing go back button, navigate to previous page.

7. Organization Editor

1. Institution (root node), departments, sub-departments, units are displayed as a hierarchy graph.
2. By pressing department node, action buttons (add sub-department, edit, delete) appear.
 1. By selecting add sub-department, sub-department registration modal pops up and sub-department can be added as a child of the department.
 2. By selecting edit, department edit modal pops up and department can be edited.
 3. By selecting delete, department can be deleted.
3. By pressing sub department node, action buttons (add unit, edit, delete) appear.
 1. By selecting add unit, unit registration modal pops up and unit can be added as a child of the sub-department.
 2. By selecting edit, sub-department edit modal pops up and sub-department can be edited.
 3. By selecting delete, sub-department can be deleted.
4. By pressing unit node, actions buttons (edit, delete) appear.
 1. By selecting edit, unit edit modal pops up and unit can be edited.
 2. By selecting delete, unit can be deleted.

8. Positions

1. Position list page
 1. Position list can be filtered by name, position code and grade.
 2. By pressing add button, navigate to position registration page.
 3. Position list has name, position code, grade columns and open details button.
 4. Position list has 10 rows in each page. Page can be switched with pagination component below the list.
2. Position registration page
 1. Position form has name, position code, grade, initial salary, raise amount, raise count and description.
 2. By pressing confirm button, form data is validated and new position is registered.
 3. By pressing cancel button, navigate to previous page.
3. Position details page
 1. Position name, code, grade, initial salary, raise amount, raise count and description are displayed.
 2. By pressing edit button, the data can be edited. By pressing confirm, changes are applied. By pressing cancel, changes are discarded,

3. By pressing delete, the position can be deleted from the system.
4. By pressing go back button, navigate to previous page.

9. Employees

1. Employee list page

1. Employee list can be filtered by name, code, affiliation, status and raise/promotion eligibilities.
2. By pressing register button, navigate to employee registration page.
3. By pressing import button, navigate to import employee page.
4. By pressing export button, export modal pops up. By selecting the format (CSV or excel) and pressing export, the employee data can be exported as a file.
5. Employee list has name, code, affiliation, status, email columns and open details button.
6. Employee list has 10 rows in each page. Page can be switched with pagination component below the list.

2. Employee registration page

1. Employee form has first name, last name, birth date, employee code, status, affiliation, degree, position, raise count, last raise date, email, phone number, address (country, state, city, line 1, line 2, postal code) and remarks.
2. By pressing confirm, the form data is validated and new employee is registered.
3. By pressing cancel, navigate to the previous page.

3. Import employee page

1. By clicking or drag & drop files (CSV or excel) to the upper area, employee data can be imported into the memory (not yet registered).
2. By pressing clear button, data imported into the memory can be cleared.
3. By pressing CSV template button, CSV template can be downloaded.
4. By pressing Excel template button, excel template can be downloaded.
5. Imported data is displayed as a table.
6. By pressing cancel, navigate to the previous page.
7. Register all button is enabled only when there is at least 1 record imported. By pressing this button, imported records can be registered to the system.

4. Employee details page

1. First name, last name, birth date, employee code, status, affiliation, position, base salary, last raise date, current raise count, email, phone number, address (country, state, city, line 1, line 2, postal code) and remarks are displayed.
2. By pressing edit button, the following data can be edited except for the followings: position, base salary, last raise date and current raise count. By pressing confirm, the changes are applied. By pressing cancel, the changes are discarded.
3. By pressing delete button, the employee can be deleted from the system.

4. By pressing go back button, navigate to previous page.
5. If the employee's raise eligibility is met, confirm raise button is displayed. By pressing this button, raise can be applied. Otherwise, the button is hidden and next raise schedule is displayed.
6. If the employee's promotion eligibility is met, confirm promotion button is displayed. By pressing this button, confirm promotion modal pops up. By selecting next position and pressing confirm, the promotion can be completed. Otherwise, the button is hidden and next promotion schedule is displayed.

10. Commendations and Sanctions

1. Commendation/sanction list page

1. List can be filtered by title and category (commendation/sanction).
2. By pressing issue button, navigate to commendation/sanction registration page.
3. List has 10 rows in each page. Page can be switched with pagination component below the list.
4. List has title, category, issued date columns and open details button.

2. Commendation/sanction registration page

1. Commendation/sanction form has title, category, adjustment (in months), description and affected employees.
2. By pressing confirm button, the letter can be issued.
3. By pressing cancel button, navigate to the previous page.

3. Commendation/sanction details page

1. In the details tab, the title, category, adjustment, issued date, employees and description are shown.
2. In the PDF view tab, the letter is displayed as a PDF file in a viewer. The letter can be printed or downloaded as PDF.
3. By pressing go back button, navigate to the previous page.

11. Performance Evaluations

1. Performance evaluation list page

1. The list can be filtered by title, evaluator employee and evaluated employee.
2. By pressing add button, navigate to performance evaluation registration page.
3. The list has title, evaluator employee, evaluated employee, date columns and open details page.
4. The list has 10 rows in each page. Page can be switched with the pagination component below the list.

2. Performance evaluation registration page

1. Performance evaluation form has evaluator employee, evaluated employee, title, score and description.

2. By pressing confirm button, performance evaluation can be issued.
3. By pressing cancel, navigate to the previous page.
3. Performance evaluation details page
 1. In the details tab, the evaluator employee, evaluated employee, title, score and description are displayed.
 2. In the PDF view tab, the performance evaluation is displayed as a PDF file in a viewer. It can be printed or downloaded as PDF.
 3. By pressing go back button, navigate to the previous page.

12. Audit Logs

1. The audit log list can be filtered by target entity, and target ID.
2. The list has 10 rows in each page. Page can be switched with the pagination component below the list.
3. The list has user name, category, target, target ID, timestamp columns and open difference view button.
4. By pressing open difference view button, the difference between two snapshots of the modified entity is shown in JSON.

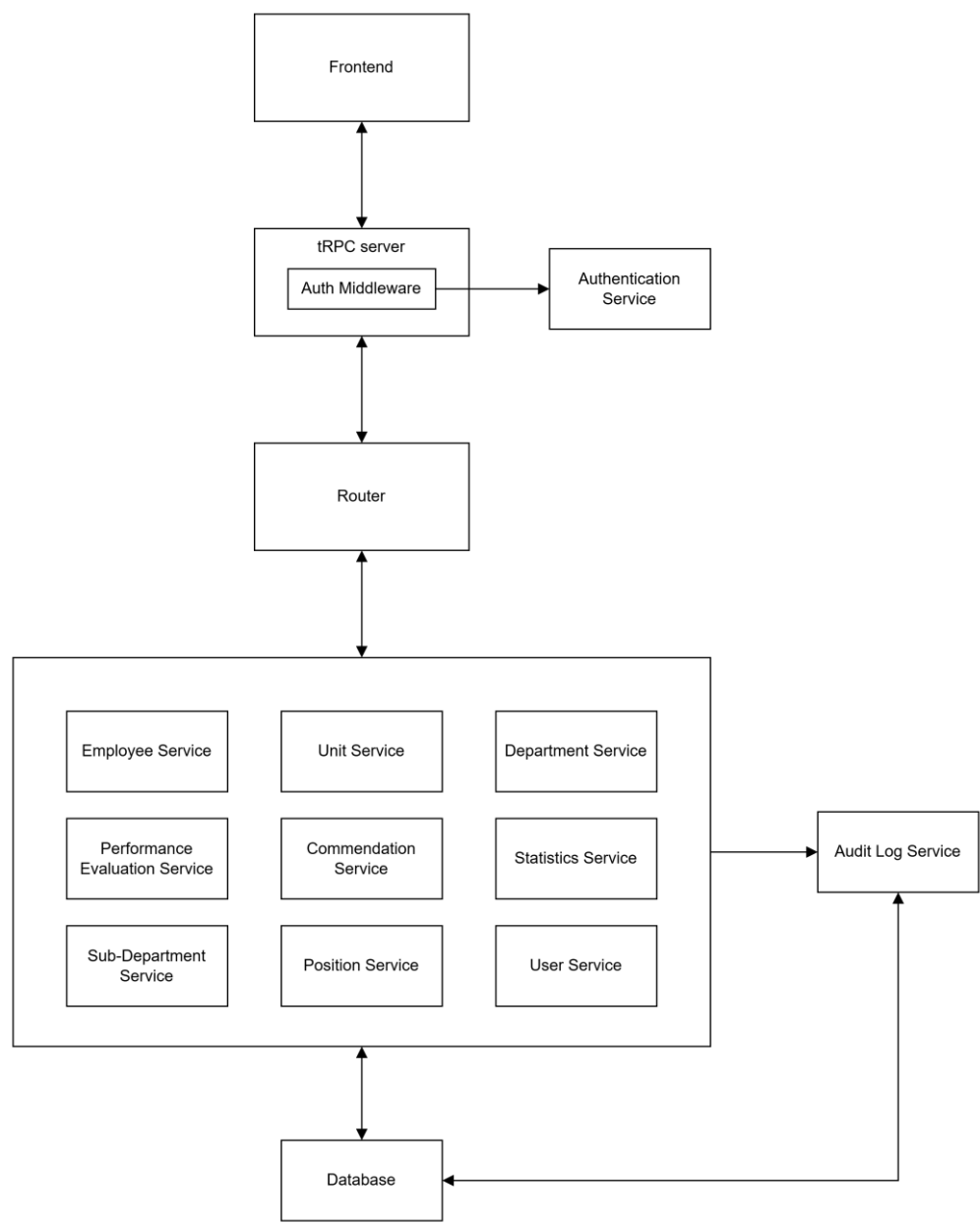
Tech Stack

Technology	Purpose, Usage	Reason
SQLite	Database	SQLite is suitable for desktop application.
Electron + Vite	Desktop application framework	By using web technologies, platform-independent applications can be developed easily. Typescript can be used to develop frontend and backend with type-safety. Better security by separating frontend process and backend process
React	Frontend development	React ecosystem allows us to integrate enterprise-ready component libraries into the application for efficiency, maintainability and better UX.
react-router-dom	Page routing in React	Easy to use in React. Suitable for not only web applications, but also desktop applications.
tRPC	Backend development	E2E type safety of API
Tanstack Query	API call	Easy of state, retry, timeout, cache management
Drizzle ORM	Database access, migration	Simple queries are auto generated. Complex queries can be written using query builder.
Zod	Data validation	Many built-in types such as emails and phone numbers. Runtime validation & type inference during development. Both for frontend and backend. Better compatibility with tRPC.
Ant Design	UI components	Suitable for enterprise applications. High customizability. Good UX. Simple and modern appearance.
i18next	Languages	Better compatibility with react. Better performance with dynamic loading. Easy to use.
tsyringe	DI container	Automation of dependency injection.

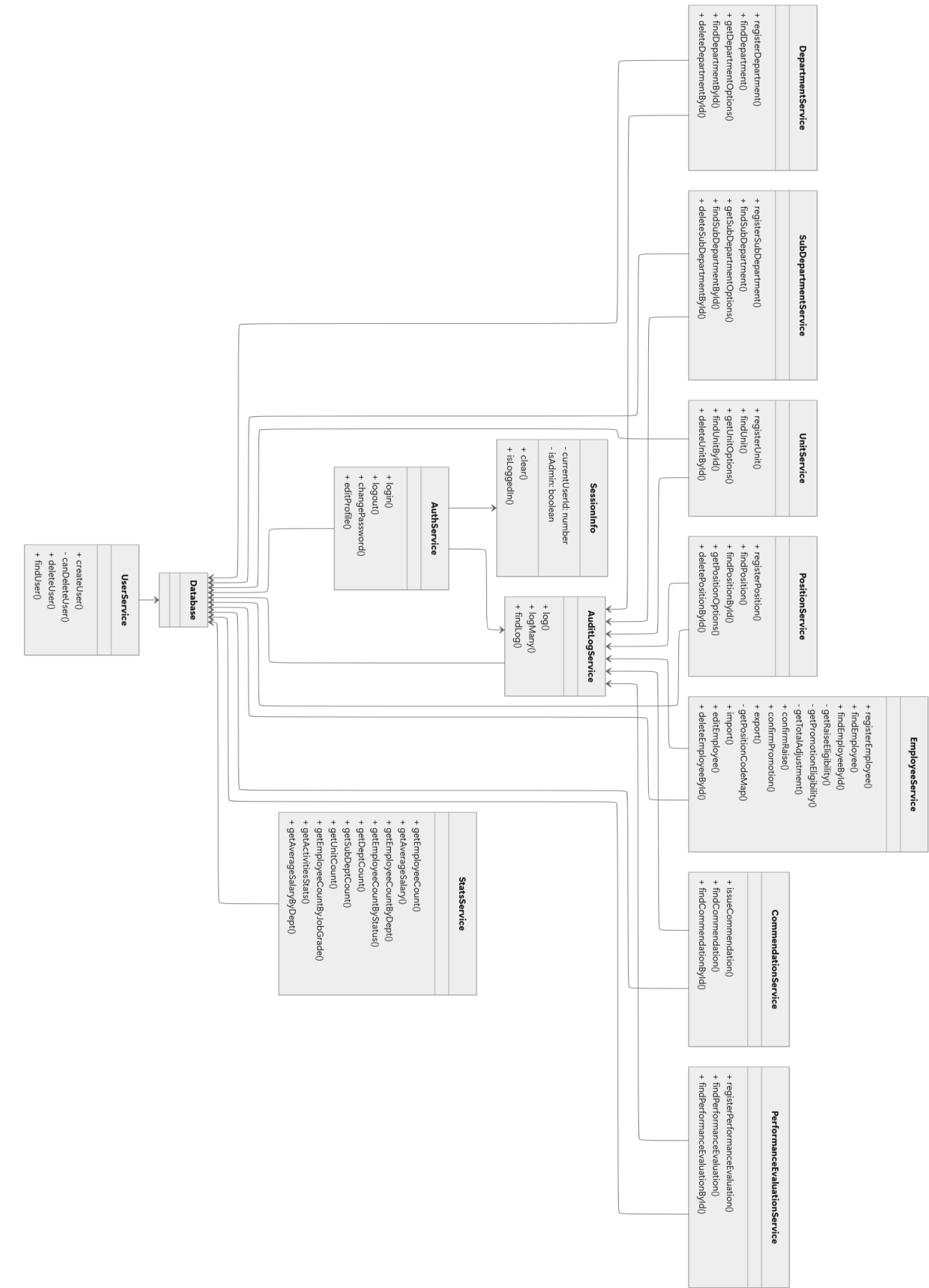
		Easy to use.
bcryptjs	Password hash generation	Good security with salting. No C++ native module is needed, hence better compatibility with Electron.
react-pdf	PDF generation & display	Easy to use in React.
xy-flow	Graph viewer	Easy to use in React. High customizability and performance. Modern appearance.
dagre	Graph layout computation	Easy to use and combine with xy-flow.
date-fns	Date manipulation	Easy to use. High performance.
fast-csv	CSV file manipulation	
react-diff-viewer-continued	Diff viewer in React	Easy to use in React. Modern appearance. Dark mode support.
xlsx	Excel file manipulation	
recharts	Chart components for React	Moder appearance. High customizability. Easy to use in React.

System Architecture Overview

Architecture Diagram

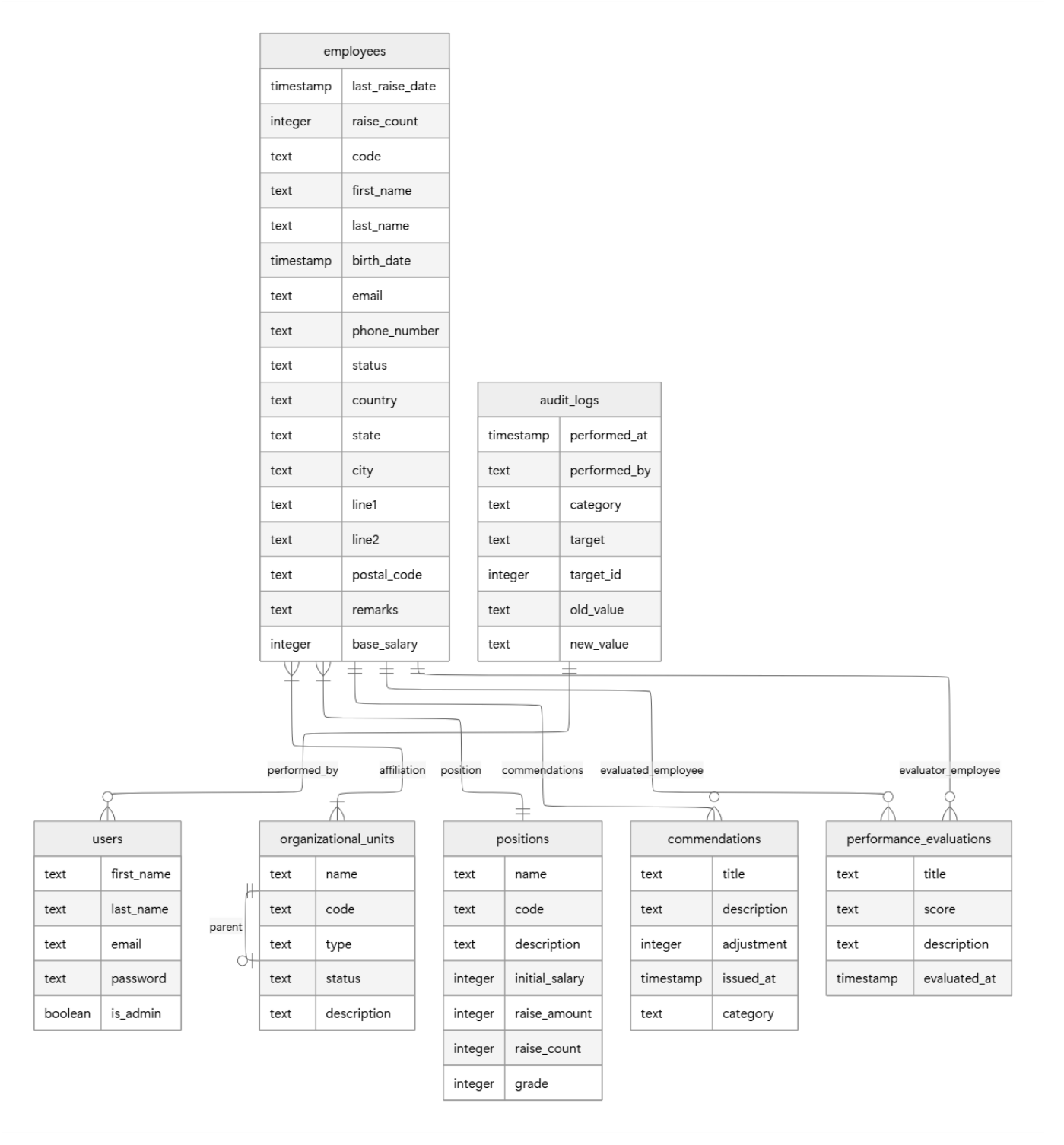


Class Diagram



Database Design

ER Diagram



Database Schema Details

users

This table stores all admin/non-admin user information of the system. When system is launched for the first time, default admin user ([admin@example.com](#)) and non-admin user ([user1@example.com](#)) is generated automatically.

Column	Type	Note
id	integer	Primary key, auto increment.

first_name	text	First name of the user.
last_name	text	Last name of the user.
email	text	Email address of the user. Unique. Used to login to the system.
password	text	Password of the user. Hashed with bcryptjs with salting for security.
is_admin	integer (0, 1)	0 for non-admin users, 1 for admin users.

organizational_units

This table stores departments, sub-departments and units all together. These different entities are stored together to make it easy to manage the hierarchical structure of the organization with self-relation of the table. These entities are differentiated by the “type” column.

Column	Type	Note
id	integer	Primary key, auto increment.
name	text	Name of the entity.
code	text	Code (unique ID) of the entity. Used in employee data importing instead of the primary key for better UX.
type	text (“DEPARTMENT”, “SUB_DEPARTMENT”, “UNIT”)	Type of the entity. Used to differentiate entity type. Mapped to Enum by drizzleORM.
status	text (“ACTIVE”, “SUSPENDED”, “CLOSED”)	Status of the entity. Mapped to Enum by drizzleORM.
parent_id	integer	Foreign key that references the table itself. Used to store organization hierarchy information. For departments, this column should be NULL.
description	text	Description of the entity.

positions

This table stores positions that can later be used to register employees and automate raise/promotion eligibility and salary computations.

Column	Type	Note
id	integer	Primary key, auto increment.
name	text	Name of the position.
code	text	Code (unique ID) of the position. Used in employee data importing instead of the primary key for better UX.
description	text	Description of the position.
initial_salary	integer	Initial salary of the position. Starting from this

		value, the salary increases by the specified raise_amount every year.
raise_amount	integer	Amount of raise in each year.
raise_count	integer	Number of raise required to become eligible for promotion to the next job grade.
grade	integer	Job grade of the position. Between 1 (highest) to 12 (lowest).

employees

This table stores all employees registered in the system.

Column	Type	Note
id	integer	Primary key, auto increment.
position_id	integer	Foreign key that references the position table.
last_raise_date	integer (timestamp)	Timestamp of the last raise date. Used to compute raise/promotion eligibility and next raise/promotion schedule. This column should be updated upon every raise/promotion.
raise_count	integer	The number of raises the employee has completed in the current position.
organization_id	integer	Foreign key that references organizational_units table. This column is also referred to as “affiliation” of the employee in the application.
code	text	Code (unique ID) of the employee.
first_name	text	First name of the employee.
last_name	text	Last name of the employee.
birth_date	integer (timestamp)	Birth date of the employee.
email	text	Email address of the employee.
phone_number	text	Phone number of the employee.
status	text (“ACTIVE”, “SUSPENDED”, “ON_LEAVE”, “SICK_LEAVE”, “PARENTAL_LEAVE”, “NOTICE_PERIOD”, “TERMINATED”)	Status of the employee. Mapped to Enum by drizzleORM.
country	text	Country (address) of the employee.
state	text	State (address) of the employee.
city	text	City (address) of the employee.

line1	text	Line 1 (address) of the employee.
line2	text	Line 2 (address) of the employee.
postal_code	text	Postal code (address) of the employee.
remarks	text	Remarks on the employee.
base_salary	integer	Base salary of the employee. Automatically calculated based on the employee's position and the raise count. Updated upon every raise/promotion.

commendations

This table stores all commendation letters and sanction letters altogether, although it is named “commendations” for simplicity. Different types (commendations or sanctions) are differentiated with “category” column.

Column	Type	Note
id	integer	Primary key, auto increment.
title	text	Title of the commendation/sanction letter.
description	text	Description (content) of the letter.
issued_at	integer (timestamp)	Issued date.
category	text (“COMMENDATION”, “SANCTION”)	Category of letters. This column is used to differentiate commendations and sanctions. Mapped to Enum by drizzleORM.
adjustment	integer	Number of adjustment months that should be accounted for when computing raise/promotion eligibility. It should be a positive value for both commendations and sanctions.

performance_evaluations

This table stores the performance evaluations of employees.

Column	Type	Note
id	integer	Primary key, auto increment.
title	text	Title of the performance evaluation.
score	text	Overall evaluation score. The values could be, for example, A, B, C or on a scale from 1 to 5 (up to the user).
description	text	Description (content) of the performance evaluation.
evaluated_at	Integer (timestamp)	Date of evaluation.

evaluator_employee	text	Name of the evaluator employee. Only the name is stored (not a foreign key to the employee) so that even after the employee is deleted, data can be retained.
evaluated_employee	text	Name of the evaluated employee. Only the name is stored (not a foreign key to the employee) so that even after the employee is deleted, data can be retained.

audit_logs

This table stores the logs of the system for auditing.

Column	Type	Note
id	integer	Primary, auto increment.
performed_at	Integer (timestamp)	The timestamp of the log.
performed_by	text	The name of the user who performed the action. Only the name of the stored (not a foreign key to users table) so that even after the user is deleted, data can be retained.
category	text (“AUTH”, “CREATE”, “EDIT”, “DELETE”)	Type of the action performed. Mapped to Enum by drizzleORM.
target	text (“DEPARTMENT”, “SUB_DEPARTMENT”, “UNIT”, “POSITION”, “EMPLOYEE”, “COMMENDATION”, “PERFORMANCE_EVALUATION”)	Target entity of the action. Mapped to Enum by drizzleORM.
target_id	integer	Target ID of the action.
old_value	text	Snapshot of the target entity before the action. Stored in JSON format. In case of “CREATE” or “AUTH”, this is NULL. In case of “EDIT”, this is the value before modification. In case of “DELETE”, this is the value before deletion.
new_value	text	Snapshot of the target entity after the action. Stored in JSON format. In case of “DELETE” or “AUTH”, this is NULL. In case of “EDIT”, this is the value after modification. In case of “CREATE”, this is the value that has just been inserted.

Implementation Details

Frontend

- Related files:
 - src/renderer/public (Resources)
 - src/renderer/src/* (React components and pages)

tRPC server

- Related files:
 - src/main/index.ts (Start server process)
 - src/main/trpc.ts (Auth Middleware definition & registration)

Router

- Related files:
 - src/main/router.ts (Aggregating routers for each service)
 - src/main/modules/*//*.router.ts (Router for each service)

Database

- Related files:
 - src/main/db/index.ts (Database configuration and client instantiation)
 - src/main/db/schema.ts (Schema definition)
 - src/main/db/seed.ts (Generating default users)

User Service

- Related files:
 - src/main/modules/users/user.service.ts

Method	Requirements
createUser	1.4.1, 1.4.2
canDeleteUser	1.3.5
deleteUser	1.3.5
findUser	1.3.1, 1.3.3, 1.3.4

Department Service

- Related files:
 - src/main/modules/departments/department.service.ts

Method	Requirement
--------	-------------

registerDepartment	4.2.1, 4.2.2, 4.3.2
findDepartment	4.1.1, 4.1.3, 4.1.4
getDepartmentOptions	5.1.1, 5.2.1, 9.1.1, 9.2.1
findDepartmentById	4.3.1
deleteDepartmentById	4.3.3
findAll	7.1

Sub-Department Service

- Related files:
 - src/main/modules/sub-departments/sub-department.service.ts

Method	Requirement
registerSubDepartment	5.2.1, 5.2.2, 5.3.2
findSubDepartment	5.1.1, 5.1.3, 5.1.4
getSubDepartmentOptions	6.1.1, 6.2.1, 9.1.1, 9.2.1
findSubDepartmentById	5.3.1
deleteSubDepartmentById	5.3.3

Unit Service

- Related files:
 - src/main/modules/units/unit.service.ts

Method	Requirement
registerUnit	6.2.1, 6.2.2, 6.3.2
findUnit	6.1.1, 6.1.3, 6.1.4
getUnitOptions	9.1.1, 9.2.1
findUnitById	6.3.1
deleteUnitById	6.3.3

Position Service

- Related files:
 - src/main/modules/positions/position.service.ts

Method	Requirement
registerPosition	8.2.1, 8.2.2, 8.3.2
findPosition	8.1.1, 8.1.3, 8.1.4
findPositionById	8.3.1
getPositionOptions	9.2.1, 9.4.6

deletePositionById	8.3.3
--------------------	-------

Employee Service

- Related files:
 - src/main/modules/employees/employee.service.ts

Method	Requirement
registerEmployee	9.2.1, 9.2.2
findEmployee	9.1.1, 9.1.5, 9.1.6
findEmployeeById	9.4.1, 9.4.5, 9.4.6
getRaiseEligibility	9.4.5
getPromotionEligibility	9.4.6
getTotalAdjustment	9.4.5, 9.4.6
confirmRaise	9.4.5
confirmPromotion	9.4.6
export	9.1.4
getPositionCodeMap	9.3
getAffiliationCodeMap	9.3
import	9.3
editEmployee	9.4.2
deleteEmployeeById	9.4.3

Commendation Service

- Related files:
 - src/main/modules/commendations/commendation.service.ts

Method	Requirement
issueCommendation	10.2.1, 10.2.2
findCommendation	10.1.1, 10.1.3, 10.1.4
findCommendationById	10.3.1, 10.3.2

Performance Evaluation Service

- Related files:
 - src/main/modules/performance-evaluations/performance-evaluation.service.ts

Method	Requirement
registerPerformanceEvaluation	11.2.1, 11.2.2
findPerformanceEvaluation	11.1.1, 11.1.3, 11.1.4
findPerformanceEvaluationById	11.3.1, 11.3.2

Audit Log Service

- Related files:
 - src/main/modules/audit-logs/audit-log.service.ts

Method	Requirement
log	12.3, 12.4
logMany	12.3, 12.4
findLog	12.1, 12.2

Authentication Service

- Related files:
 - src/main/modules/auth/auth.service.ts

Method	Requirement
login	1.1.2
logout	1.2.4
getMe	1.2.3
changePassword	1.2.5
editProfile	1.2.6

Future Improvements

Encryption of sensitive employee data

Sensitive employee information such as salaries, contact details, and personal records should be encrypted to improve security and privacy protection. A possible implementation is encrypting selected database fields or using operating-system-level secure storage for encryption keys. One challenge is balancing security with application performance and ensuring that encrypted data can still be searched or processed efficiently.

Fine-grained Role-Based Access Control

The current authorization system can be extended to support more detailed permissions for different user roles. For example, managers may view employee records but only HR administrators can modify salary information. This would improve security and organizational flexibility. The main challenge is designing a permission structure that remains maintainable as the number of roles and permissions increases.

Desktop notification

Desktop notifications could improve usability by informing users about important events such as pending promotions and raises. This can be implemented using Electron's notification API or platform-specific notification systems. Challenges include preventing excessive notifications and ensuring consistent behavior across different operating systems.

UI/UX improvement

The user interface can be further improved to make the application more intuitive and visually consistent. Potential improvements include responsive layouts, improved navigation, accessibility support, and clearer data visualization. The challenge is balancing visual complexity with usability while maintaining performance in a desktop environment.

Bulk raise/promotion confirmation

The application could support confirming multiple salary raises or promotions simultaneously to improve administrative efficiency. This feature would reduce repetitive manual operations for HR staff. Challenges include designing a safe confirmation workflow, preventing accidental updates, and ensuring that large batch operations remain consistent in the database.

Implementing soft-delete and recovery

Currently, deleted records may be permanently removed from the database. Introducing a soft-delete mechanism would allow records to be marked as deleted instead of being erased immediately, enabling recovery when necessary. The main challenge is modifying database queries so that deleted records are handled correctly without affecting application performance.

Database backup

Adding an automated backup system would reduce the risk of data loss caused by system failures or accidental modifications. A possible approach is scheduled local backups or exporting encrypted backup files. Challenges

include ensuring backup consistency, minimizing storage usage, and providing a simple recovery process for users.

Refactoring

As the project grows, refactoring the codebase would improve maintainability, readability, and scalability. Examples include separating business logic more clearly, reducing duplicated code, and improving module structure. The challenge is performing refactoring without introducing regressions or breaking existing functionality.